

ADR

Realm Membership and Identity Model

Version 1.4.2-SNAPSHOT, 2026-09-21T22:14:39Z

Table of Contents

1. Status	1
2. Context	2
3. Decision	3
4. Consequences	5

Chapter 1. Status

Accepted (2026-06-11) — implemented incrementally on the integration branch.

Chapter 2. Context

The framework had three half-articulated concepts: realms (a Mongo database with a catalog entry), DataDomain (org/account/tenant/owner/segment stamps on every document), and RealmTenantMembership (an onboarding record). What was undefined: what **delineates** a realm, how multiple orgs collaborate inside one realm, and how one user holds different roles in different realms. The a multi-tenant platform two-plane split and the fulfillment-style collaboration model (carrier/supplier/customer orgs against shared sales orders and fulfillment plans) both need these answers to be explicit.

Chapter 3. Decision

1. **Realm = placement + lifecycle.** A realm selects a logical Mongo database (and possibly its own connection or cluster) and owns migration, index, seed, backup, residency, and destructive lifecycle. A dedicated realm has one tenant OWNER. A pooled realm has one platform/operator OWNER plus many tenant PARTICIPANT memberships. When you ask "what can be dropped/backed up/migrated as one unit", the answer is a realm, never an individual participant in a pooled realm. `Realm.deploymentType` persists this invariant as `DEDICATED` or `SHARED`. Missing legacy values resolve to `DEDICATED`; only an explicit `SHARED` value permits pooled admission.
2. **DataDomain = visibility.** Org/account/tenant/owner/segment stamps plus the policy engine (row filters, relationship policies via `hasEdge`, field-level exclusions) control **who sees what inside** a realm. Placement never substitutes for policy, and policy never substitutes for placement.
3. **Orgs/accounts are MEMBERS of realms, with exactly one OWNER.** `RealmTenantMembership.membershipRole`: the OWNER provisioned the realm and holds lifecycle/billing authority; PARTICIPANTs are collaborating orgs whose visibility comes entirely from DataDomain + policies. The `POST /admin/tenants` defaults to `DEDICATED_REALM` (one tenant OWNER) and accepts `POOLED_REALM` to deliberately admit a PARTICIPANT to an existing realm whose persisted deployment type is `SHARED`.
4. **Users have ONE global identity and N realm-role assignments.** The credential (system realm) is the identity — the GitHub account. `UserRealmRole` documents are the org-membership analog: (userId, realm, roles[], sponsoring org, status). ADMIN in realm A / VIEWER in realm B is two documents. `credential.roles[]` keeps meaning "roles in the default realm" for compatibility. Invitations reuse `AccessInviteResource`.
5. **The IdP unions assignments into per-realm claims.** At token issuance the control plane resolves `RealmMembershipService.rolesForUser(user, realm)`; the tenant plane routes by claims with no callback (JWKS validation). Embedded mode resolves the same service locally.
6. **Tenant decommission follows placement topology.** A dedicated tenant is decommissioned by revoking access, verifying a whole-database archive, and dropping its realm database. A pooled tenant is decommissioned by revoking access, verifying an export selected by the canonical DataDomain tuple (orgRefName, accountNum, tenantId, dataSegment), and assigning one purge batch plus one `purgeAfter` timestamp to all of its rows. Mongo TTL cleanup may drain collections asynchronously; the access revocation is the externally atomic barrier, and per-collection count reconciliation is the completion gate. The pooled workflow never drops or migrates the shared realm. Revocation must cover already-issued sessions/tokens as well as future authentication; suspending only the catalog membership is not a verified barrier for stateless tenant planes. TTL indexes are installed during realm initialization and verified or created idempotently by the expiration provider before stamping. Blob payloads participate in archive and expiration coverage. Before planning the purge, the runtime reconciles every existing user collection against an explicit `TENANT_SCOPED`, `REALM_SHARED`, or `SYSTEM_GLOBAL` declaration. Unclassified collections fail closed; only existing collections declared `TENANT_SCOPED` enter the purge plan.
7. **Pooled admission is metadata-only.** The target realm must already be migrated, indexed, and seeded. Admitting a participant creates the tenant membership and requested administrator assignment, but does not run migrations, base seeds, archetypes, or compatibility/demo identity

setup. A user has at most one active tenant DataDomain assignment in a realm until token issuance supports explicit tenant selection.

Chapter 4. Consequences

- The fulfillment collaboration model is expressible without new isolation machinery: one realm, one owner org, N participant orgs, relationship field policies shaping each org's slice.
- Cross-realm queries stay forbidden; anything cross-realm goes through the control plane's denormalized registry (design review §2.2).
- Phase C's control-plane API must expose membership resolution alongside the realm catalog (same seam, same fail-loud remote behavior).
- GitHub-model mapping for product docs: account → credential, org → org, org membership → UserRealmRole, repo → realm, repo role → roles[] per realm, invitation → AccessInvite, personal default context → credential.defaultRealm.